第4章 第9节 Prompt基础与必要场景

- 来源: https://class.imooc.com/lesson/5270#mid=104910
- 阶段: 阶段二: 【实战入门】大模型开发与知识接入
- 周: 第4周 理解大模型开发
- 模块: Prompt基础与必要场景
- 课节: 图文 1-1 Prompt基础与必要场景

1-1 Prompt基础与必要场景

第4章 第9节 Prompt基础与必要场景

> [!TIP] 阅读指南 上一节学习了API参数控制。但参数调得再好，如果提示词写不清楚，AI的回答仍然可能不尽如人意。本节从基础开始——理解什么是提示词、提示词工程，以及真正需要它的场景。

9.1 什么是提示词（Prompt）

提示词，就是对AI说的话

我们之前在API调用案例里已经用到过提示词，只是没有明确它叫"提示词（Prompt）。提示词的质量，直接决定了AI回答的质量。

```
messages = [
    {"role": "user", "content": "把这句话翻译成英文"}  # ←content 这就是提示词
]
```

想象你在跟一个非常聪明但不了解你的助手对话：

- 你说"翻译这段文字"——助手虽然知道要翻译，但不知道翻译成什么语言、什么风格
- 你说"把这段文字翻译成英文，要专业严谨的风格，只给我翻译结果"——助手立刻明白了所有要求

第一句话就是糟糕的提示词，第二句话就是优秀的提示词。

糟糕的提示词

优秀的提示词

提示词并非越多越好

既然第二句话更详细，那提示词是不是越多越好？未必。

未必。

提示词设计其实是一门平衡的艺术：给太少，AI不知道你要什么，可能答非所问；给得刚好，清晰、精准，AI精准命中需求；给太多，不只是消耗Token多，过多的内容反而会干扰AI思考，让质量下降。

一个反例

```
# 看似详细，实则冗余的提示词
# 重复表达、优先级不清、缺乏结构
"""请帮我翻译下面这段文字。我需要的是英文翻译，翻译的时候请注意语法的准确性，
同时也要考虑语义的完整性，另外在翻译过程中如果遇到专业术语要用专业的表达方式，
还有就是不要添加多余的解释说明，因为我只需要翻译结果就可以了，
格式上也要保持简洁，不需要额外的排版和修饰。"""

# 简洁高效的提示词
# 优点：结构清晰、重点突出、易于理解
"""把这段文字翻译成英文：
- 专业严谨的风格
- 只返回翻译结果，不要解释"""
```

好的提示词特征

- 明确但不啰嗦：说清楚要求，但不过度解释
- 结构化表达：用列表、分点，让AI快速抓住重点
- 关键信息优先：最重要的要求放在前面
- 避免歧义：用具体描述代替模糊词汇

在接下来的内容中，我们会系统学习这些技巧。

9.2 什么是提示词工程（Prompt Engineering）

既然提示词这么重要，如何设计出优秀的提示词，就成了一门技术——这就是提示词工程。

提示词工程包括：

- 如何清晰地表达需求

- 如何提供必要的背景信息
- 如何规范输出格式
- 如何引导AI的思考过程

初级提示词

```
写一个排序函数

1
```

过于简单，AI回复容易跑偏。

优化后

```
写一个快速排序函数，包含注释

1
```

AI给你：

```
def quick_sort(arr):
    """快速排序实现"""
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]  # 选择中间元素作为基准
    left = [x for x in arr if x < pivot]    # 小于基准的元素
    middle = [x for x in arr if x == pivot] # 等于基准的元素
    right = [x for x in arr if x > pivot]   # 大于基准的元素
    return quick_sort(left) + middle + quick_sort(right)
```

有注释更容易理解，这是最初级的提示词工程——明确说明要求。这是基础，还远远谈不上"提示词工程"。

研究表明，好的提示词能让普通模型超越顶级模型，且优化成本远低于模型训练或参数调优。

这就是为什么提示词工程成为了AI应用开发者的必修课。

必修课

9.3 不是任何时候都需要提示词工程

不需要提示词工程的场景

如果你只是想：

- 向AI问知识：“什么是RESTful API？”
- 闲聊天：“今天天气怎么样？”
- 获取灵感：“给我一些项目想法”

直接问就好。过度设计提示词反而是累赘：

```
# 过度设计（没必要）
"""你是一位资深的软件架构师，拥有20年的行业经验，擅长Web开发。
请用通俗易懂的语言，结合实际例子，详细解释以下概念...

问题：什么是RESTful API？
"""

# 直接问（够用）
"什么是RESTful API？用例子说明一下"
```

原因很简单

AI本身就很擅长解释知识，复杂的提示词反而增加思考成本，容易产生不相干的"废话"。

这种场景下，自然对话才是最好的方式。

自然对话才是最好的方式

必须使用提示词工程的场景

一旦需要精确控制AI的输出，提示词工程就变得至关重要。典型场景包括：

精确控制AI的输出

- 场景1：多模态生成

当使用AI生成图像、视频、音频时，提示词工程的重要性成倍放大：

```
# 图像生成（如Midjourney、DALL-E、Stable Diffusion）
糟糕Prompt："一只猫"
→ 得到普通照片，风格不可控

优秀Prompt："一只蓬松的橙色猫坐在窗台上，黄昏时分的光线，电影级构图，景深效果，4K画质，照片级真实感"
→ 得到电影级画质，风格精准

# 视频生成（如Sora、Runway）
糟糕Prompt："海滩日落"
→ 画面普通，镜头运动不可控

优秀Prompt："宁静海滩的日落场景，慢速跟随镜头，海浪轻轻翻滚，温暖的橙色天空，镜头从左向右移动，24帧率，电影感"
→ 镜头语言、节奏、氛围全部可控
```

有的人用AI一张满意的图片都很难生成，但有的人确可以用AI进行漫画、电影的创造。差距主要还是在提示词塑造能力上。

多模态生成对Prompt的依赖

特点	文本生成	多模态生成
可调整性	生成后可以再改	生成后几乎无法修改
成本	几分钱/次	几块钱/次，试错成本高
生成时间	几秒	几分钟到几十分钟
结果随机性	较小	非常大，同一Prompt生成10次都不同

多模态的生成和修改成本更高。越是这样，提示词工程就越重要。

- 场景2：代码生成

生成代码时，细微的Prompt差异会导致结果大相径庭：

> 写一个API调用函数

```
import requests

def call_api(url):
    response = requests.get(url)
    return response.json()
```

可能不符合项目规范；没有错误处理、重试、日志。

```
使用httpx库编写一个HTTP API调用函数，要求：
1. 支持GET/POST方法
2. 超时时间30秒
3. 失败重试3次，指数退避
4. 记录详细日志（请求/响应/错误）
5. 返回JSON数据，异常时抛出自定义异常
6. 添加类型注解和docstring
```

```
import httpx
#......
logger = logging.getLogger(__name__)

@retry(stop=stop_after_attempt(3), wait=wait_exponential(multiplier=1, min=4, max=10))
def call_api(url: str, method: str = "GET", data: Dict[str, Any] = None) -> Dict[str,
Any]:
    """
    HTTP API调用函数，支持重试和日志记录
    Args:
        url: API地址
    Returns:
        JSON响应数据
    Raises:
        APIError: API调用失败
    """
    try:
        logger.info(f"请求 {method} {url}, data={data}")

    except httpx.HTTPError as e:
        logger.error(f"API调用失败: {e}")
        raise APIError(f"API调用失败: {e}") from e
```

代码包含完整的错误处理、重试、日志、类型注解。

- 场景3：应用系统集成

当把大模型集成到应用系统中时，Prompt需要像接口文档一样严格定义输出：

- 数据格式（JSON、XML、CSV等）
- 字段名称（用英文还是中文？用下划线还是驼峰？）
- 字段类型（数字、字符串、布尔值、数组）
- 数据结构（嵌套层级、数组元素）
- 特殊处理（空值如何表示？日期格式？）

举个例子

传统API返回固定的JSON结构：

```
{
  "temperature": 25,
  "humidity": 60,
  "condition": "sunny"
}
```

AI API如果不用Prompt控制，返回的是自然语言：

```
今天天气不错，气温25°C，阳光明媚...

1
```

通过Prompt精确控制，可以让AI返回指定格式、字段名、类型的JSON：

```
{
  "temperature": 25,           // 数字类型，不带单位
  "humidity": 60,              // 数字类型（百分比）
  "condition": "sunny",        // 字符串，小写英文
  "feel_like": "comfortable",  // 指定字段名用下划线
  "advice": [                  // 数组类型
    "适合户外活动",
    "注意防晒"
  ]
}
```

格式细节的重要性

假设程序需要解析天气数据：

- 如果AI返回"temperature": "25度"（字符串带单位），int()转换会报错
- 如果字段名是"Temperature"而你用data["temperature"]，程序会崩溃

9.4 下一节预告

了解了需要提示词工程的场景后，接下来系统学习Prompt的五要素——角色、任务、上下文、格式、约束，这是设计优秀提示词的核心框架。

9.5 ■ 学点英语

中文	English	音标	说明
提示词	Prompt	/prɑːmpt/	用户向AI发送的指令或问题文本
提示词工程	Prompt Engineering	/prɑːmpt ˌendʒɪˈnɪrɪŋ/	设计和优化提示词以控制AI输出的技术
多模态生成	Multimodal Generation	/ˌmʌltiˈmoʊdl ˌdʒenəˈreɪʃn/	AI生成图像、视频、音频等多类型内容
代码生成	Code Generation	/koʊd ˌdʒenəˈreɪʃn/	通过自然语言描述让AI自动编写代码
应用集成	System Integration	/ˈsɪstəm ˌɪntɪˈgreɪʃn/	将大模型接入应用程序系统的开发过程